최적 k 실험

답변에 넣는 최종 문맥 수 k를 바꿔 retrieval_a · retrieval_b를 비교하고, Langfuse에서 결과를 확인하는 방법을 정리했습니다.

한눈에 보기

구성	정답률	Faithfulness	Context Recall	Context Precision	입력 토큰/문항
A · k5	0.422	0.832	0.713	0.887	2,864
A · k8	0.422	0.859	0.746	0.855	3,428
A · k10	**0.444**	0.847	0.755	0.852	3,839
B · k5	0.411	0.825	0.719	0.896	2,843
B · k8	0.478	0.834	0.767	0.877	3,295
B · k10	**0.522**	0.846	0.778	0.845	3,628

세 난이도 합계 (구성마다 90문항, hybrid + rerank). 정답률의 굵은 글씨는 같은 검색기 안에서 가장 높은 값.

- **B는 k를 늘릴수록 좋아집니다.** 정답률 0.41 → 0.48 → 0.52 (90문항 중 10문항 더 맞힘), Faithfulness 0.825 → 0.846. 세 난이도 모두 k10이 k5보다 좋거나 같습니다.
- **A는 정답률이 거의 그대로**(0.42 → 0.42 → 0.44)이고, Faithfulness는 k8에서 가장 높습니다(0.859).
- k를 늘리면 Context Recall은 오르고 Context Precision은 내려갑니다. 필요한 내용이 더 들어오지만 쓸모없는 문맥도 함께 섞입니다.
- 입력 토큰은 k10에서 약 +30%, 응답 시간 중앙값은 0.5~1초 늘었습니다.
- advanced는 k와 관계없이 정답률 0.13~0.23에 머뭅니다. k보다 질문 분해·되묻기 판정·리랭커 쪽의 문제입니다.

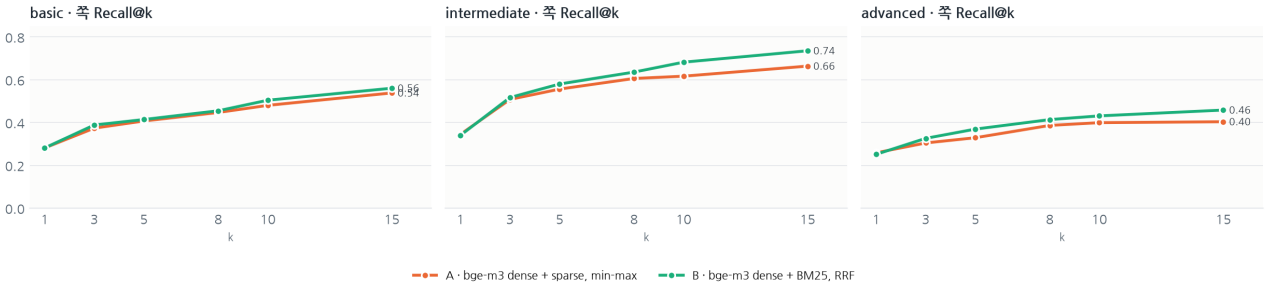
> **추천 B는 k=10, A는 k=8.** B는 정답률·Faithfulness가 함께 가장 높고, A는 정답률이 비슷한 가운데 Faithfulness가 가장 높고 토큰이 적습니다. B는 k10까지 계속 오르고 있어 k15를 추가로 확인할 만합니다.

실험 설계

	1단계: 검색만	2단계: 전체 체인
목적	후보 수·k를 넓게 바꿔 보고 2단계에서 돌릴 k를 고른다	고른 k로 실제 답변을 만들어 정답률·RAGAS·토큰·시간을 본다
k	1, 3, 5, 8, 10, 15	5 (오전 비교 실험 재사용), 8, 10
대상	naive · hybrid · hybrid + rerank, 후보 수·리랭커 후보 수 여러 개	hybrid + rerank (A 후보 500·리랭커 50 / B 후보 30·리랭커 50)
문항	난이도별 정답 근거가 있는 전체 문항	난이도별 development 30문항
지표	문서 Hit@k · Recall@k · MRR@k, 쪽 Hit@k · Recall@k, 검색 시간	정답률, Judge 4항목, RAGAS 4지표, 검색 Hit@k · Recall@k, 토큰, 응답 시간
LLM	없음 (비용 0)	답변·RAGAS 채점 gpt-6-luna, Judge는 Langfuse 서버 평가기
결과 파일	k_sweep.json	Langfuse 실험 + collected_k.json

- **정답률** = 서버 Judge 4항목(사실 facts · 결론 decision · 근거 evidence · 조건 conditions)을 모두 통과한 문항의 비율.
- k 외의 설정(프롬프트, 답변 모델, 리랭커, 가중치 0.5/0.5, Qdrant 후보 수)은 모두 같게 두었습니다. 검색 판정 k도 넣은 문맥 수와 같게 맞췄습니다.

1단계: 검색만으로 본 k



hybrid + rerank의 쪽 단위 Recall@k. 답변에 필요한 쪽(evidence_groups의 책·쪽)을 상위 k개 안에 얼마나 가져왔는지.

basic · 문서 Hit@k / 쪽 Recall@k

구성	k=1	k=3	k=5	k=8	k=10	k=15	검색 시간
A · naive	0.63 / 0.16	0.87 / 0.32	0.93 / 0.36	0.93 / 0.42	0.97 / 0.49	0.99 / 0.53	0.05초
A · hybrid	0.66 / 0.24	0.87 / 0.31	0.93 / 0.40	0.94 / 0.46	0.97 / 0.51	0.99 / 0.55	0.24초
A · hybrid_rerank	0.71 / 0.28	0.94 / 0.37	0.97 / 0.41	0.97 / 0.45	0.97 / 0.48	0.99 / 0.54	0.39초
B · naive	0.63 / 0.16	0.87 / 0.32	0.93 / 0.36	0.93 / 0.42	0.97 / 0.49	0.99 / 0.53	0.04초
B · hybrid	0.69 / 0.19	0.88 / 0.34	0.96 / 0.45	0.97 / 0.51	0.99 / 0.53	1.00 / 0.61	0.05초
B · hybrid_rerank	0.71 / 0.28	0.94 / 0.39	0.97 / 0.41	0.97 / 0.46	0.99 / 0.50	1.00 / 0.56	0.33초

intermediate · 문서 Hit@k / 쪽 Recall@k

구성	k=1	k=3	k=5	k=8	k=10	k=15	검색 시간
A · naive	0.89 / 0.29	0.96 / 0.48	0.96 / 0.51	0.98 / 0.61	0.98 / 0.63	0.98 / 0.66	0.06초
A · hybrid	0.91 / 0.30	0.98 / 0.47	0.98 / 0.54	0.98 / 0.58	0.98 / 0.61	0.98 / 0.64	0.12초
A · hybrid_rerank	0.98 / 0.34	0.98 / 0.51	0.98 / 0.56	0.98 / 0.61	0.98 / 0.62	0.98 / 0.66	0.76초
B · naive	0.89 / 0.29	0.96 / 0.48	0.96 / 0.51	0.98 / 0.61	0.98 / 0.63	0.98 / 0.66	0.05초
B · hybrid	0.96 / 0.34	1.00 / 0.55	1.00 / 0.60	1.00 / 0.66	1.00 / 0.67	1.00 / 0.73	0.40초
B · hybrid_rerank	0.98 / 0.34	1.00 / 0.52	1.00 / 0.58	1.00 / 0.64	1.00 / 0.68	1.00 / 0.74	0.58초

advanced · 문서 Hit@k / 쪽 Recall@k

구성	k=1	k=3	k=5	k=8	k=10	k=15	검색 시간
A · naive	0.40 / 0.14	0.57 / 0.24	0.67 / 0.28	0.71 / 0.31	0.74 / 0.31	0.78 / 0.32	0.03초
A · hybrid	0.55 / 0.21	0.67 / 0.29	0.79 / 0.32	0.84 / 0.34	0.86 / 0.35	0.90 / 0.36	0.09초
A · hybrid_rerank	0.78 / 0.26	0.83 / 0.30	0.88 / 0.33	0.90 / 0.39	0.90 / 0.40	0.90 / 0.40	0.44초
B · naive	0.40 / 0.14	0.57 / 0.24	0.67 / 0.28	0.71 / 0.31	0.74 / 0.31	0.78 / 0.32	0.05초
B · hybrid	0.50 / 0.19	0.81 / 0.35	0.90 / 0.43	0.90 / 0.52	0.93 / 0.55	0.98 / 0.58	0.66초
B · hybrid_rerank	0.76 / 0.25	0.84 / 0.33	0.90 / 0.37	0.90 / 0.41	0.91 / 0.43	0.95 / 0.46	1.02초

- **문서 Hit은 k=5에서 거의 꽉 찹니다.** 정답 문서는 이미 상위 5개 안에 있습니다.
- **쪽 Recall은 k를 늘릴수록 오릅니다.** 필요한 쪽이 여러 곳에 흩어져 있기 때문입니다. A advanced는 k8 이후 거의 그대로, B는 k15까지 오릅니다.
- k3 이하는 advanced 문서 Hit이 0.88 → 0.83으로 떨어져 제외, k15는 토큰이 k5의 약 3배라 2단계에서 일단 제외 → **2단계 후보 k = 5, 8, 10**.
- 리랭커 후보 수 30 · 50 · 100은 차이가 거의 없고 100은 시간만 두 배라 50을 유지합니다.

2단계: 전체 체인으로 본 k



basic · 답변 품질

구성	n	정답률	Judge 평균	Faithfulness	Context Recall	Context Precision	Answer Relevancy
A · k5	30	0.467	0.725	**0.925**	0.839	**0.942**	0.772
A · k8	30	0.533	**0.742**	0.902	0.889	0.929	0.711
A · k10	30	0.600	0.700	0.884	0.891	0.933	0.736
B · k5	30	0.500	0.642	0.887	0.831	0.939	0.760
B · k8	30	0.567	0.708	0.863	0.889	0.940	**0.794**
B · k10	30	**0.667**	0.733	0.905	**0.898**	0.941	0.783

intermediate · 답변 품질

구성	n	정답률	Judge 평균	Faithfulness	Context Recall	Context Precision	Answer Relevancy
A · k5	30	**0.667**	0.733	0.816	0.809	**0.935**	0.667
A · k8	30	0.600	0.667	0.881	0.813	0.894	**0.725**
A · k10	30	0.600	0.658	0.881	**0.865**	0.900	0.721
B · k5	30	0.567	0.683	0.867	0.811	0.904	0.694
B · k8	30	0.633	0.717	0.890	0.828	0.897	0.703
B · k10	30	**0.667**	**0.750**	**0.899**	0.849	0.865	0.721

advanced · 답변 품질

구성	n	정답률	Judge 평균	Faithfulness	Context Recall	Context Precision	Answer Relevancy
A · k5	30	0.133	**0.425**	0.754	0.490	0.783	0.514
A · k8	30	0.133	0.308	**0.794**	0.535	0.741	0.581
A · k10	30	0.133	0.292	0.776	0.508	0.724	0.551
B · k5	30	0.167	0.367	0.722	0.515	**0.846**	0.587
B · k8	30	**0.233**	0.350	0.750	0.583	0.793	**0.597**
B · k10	30	**0.233**	0.375	0.734	**0.586**	0.728	0.553

2단계: 검색 점수 · 토큰 · 응답 시간

basic

구성	Hit@k	Recall@k	입력 토큰/문항	출력 토큰/문항	응답 시간 중앙값	응답 시간 평균
A · k5	0.929	0.548	2,805	228	3.8초	4.0초
A · k8	0.964	0.642	3,174 (+13%)	246	4.4초	5.2초
A · k10	0.964	0.673	3,420 (+22%)	247	4.4초	5.3초
B · k5	0.964	0.572	2,803	227	3.5초	4.1초
B · k8	0.964	0.671	3,173 (+13%)	251	4.3초	5.5초
B · k10	1.000	0.763	3,416 (+22%)	246	4.4초	5.3초

intermediate

구성	Hit@k	Recall@k	입력 토큰/문항	출력 토큰/문항	응답 시간 중앙값	응답 시간 평균
A · k5	1.000	0.743	2,987	216	4.2초	5.1초
A · k8	1.000	0.790	3,829 (+28%)	225	5.2초	6.2초
A · k10	1.000	0.795	4,532 (+52%)	228	4.5초	5.1초
B · k5	1.000	0.743	2,968	220	4.1초	5.1초
B · k8	1.000	0.776	3,564 (+20%)	224	4.5초	6.6초
B · k10	1.000	0.804	4,076 (+37%)	233	5.1초	7.3초

advanced

구성	Hit@k	Recall@k	입력 토큰/문항	출력 토큰/문항	응답 시간 중앙값	응답 시간 평균
A · k5	0.759	0.451	2,801	241	5.1초	5.4초
A · k8	0.793	0.503	3,282 (+17%)	246	5.5초	6.1초
A · k10	0.793	0.523	3,566 (+27%)	245	5.8초	6.2초
B · k5	0.828	0.497	2,760	243	9.1초	9.2초
B · k8	0.828	0.526	3,148 (+14%)	247	5.7초	16.6초
B · k10	0.862	0.543	3,391 (+23%)	251	6.1초	14.3초

- **basic**: 두 검색기 모두 k를 늘리면 정답률이 오릅니다 (A 0.47 → 0.60, B 0.50 → 0.67). A는 k가 커질수록 Judge '조건' 점수가 떨어졌습니다 (0.93 → 0.73).
- **intermediate**: B는 k10에서 정답률 0.67로 가장 높고, A는 k5(0.67)가 가장 높습니다. A의 Faithfulness는 k8부터 0.82 → 0.88로 올랐습니다.
- **advanced**: B는 k8부터 정답률 0.17 → 0.23, A는 0.13 그대로입니다. Context Recall은 둘 다 올랐지만 답변 상태 대부분이 되묻기·보류입니다.

읽을 때 30문항이라 1문항이 0.033입니다. 난이도 하나만 보면 차이 대부분이 1~4문항이라, 판단은 세 난이도 합계(90문항)를 기준으로 했습니다.

주의 B advanced는 응답 시간 평균이 14~17초로 튀었습니다(중앙값 6초). 몇 문항이 Qdrant 재시도로 오래 걸린 것으로 보입니다. A의 k5 실험은 오전 설정이라 Qdrant 후보 수가 지금(500)과 달랐을 수 있습니다.

Langfuse에서 확인하는 방법

서버 Langfuse(4.76)에서는 예전의 Runs가 **Experiments**로 바뀌었습니다. 실험 하나 = 데이터셋 30문항을 한 구성으로 돌린 결과입니다.

실험 이름 읽는 법

```
b_hybrid_rerank_20261112-1711_basic-dev-k10-gpt6luna
 └ b_ = retrieval_b (없으면 retrieval_a) · hybrid_rerank = 모드 · 20261112-1711 = 시작 시각
 · basic = 난이도 · dev = development 데이터 · k10 = 최종 문맥 수 · gpt6luna = 답변 LLM
```

난이도	k5 (기준)	k8	k10
basic	hybrid_rerank_20261112-1017_cmp_qdrant b_hybrid_rerank_20261112-1023_cmp	hybrid_rerank_20261112-1641_basic-dev-k8-gpt6luna b_hybrid_rerank_20261112-1650_basic-dev-k8-gpt6luna	hybrid_rerank_20261112-1700_basic-dev-k10-gpt6luna b_hybrid_rerank_20261112-1711_basic-dev-k10-gpt6luna
intermediate	hybrid_rerank_20261112-1027_cmp_qdrant_intermediate b_hybrid_rerank_20261112-1035_cmp_intermediate	hybrid_rerank_20261112-1721_intermediate-dev-k8-gpt6luna b_hybrid_rerank_20261112-1732_intermediate-dev-k8-gpt6luna	hybrid_rerank_20261112-1742_intermediate-dev-k10-gpt6luna b_hybrid_rerank_20261112-1754_intermediate-dev-k10-gpt6luna
advanced	hybrid_rerank_20261112-1039_cmp_qdrant_advanced b_hybrid_rerank_20261112-1252_cmp_advanced_rerun	hybrid_rerank_20261112-1807_advanced-dev-k8-gpt6luna b_hybrid_rerank_20261112-1818_advanced-dev-k8-gpt6luna	hybrid_rerank_20261112-1831_advanced-dev-k10-gpt6luna b_hybrid_rerank_20261112-1843_advanced-dev-k10-gpt6luna

위가 A, 아래가 B. 오전 실험 중 _cmp_local · _cmp_qdrant · _cmp가 붙은 naive · hybrid 실험과 ragastest 등은 k 비교와 관계없습니다.

1. 실험 목록에서 최종 점수 보기

1. 왼쪽 메뉴 **Datasets** → **shipping-basic-development-v1** (난이도별로 따로 있음) → 위쪽 **Experiments** 탭.
2. 검색창에 **dev-k**를 넣으면 이번 k 실험만 보입니다.
3. 표 오른쪽 위 **Columns**에서 보고 싶은 열을 켭니다 (아래 표).

묶음	컬 열	뜻
실험 평균	accuracy_mean	정답률 (Judge 4항목 모두 통과한 문항 비율)
실험 평균	judge_facts_mean · judge_decision_mean · judge_evidence_mean · judge_conditions_mean	Judge 항목별 평균 ([해당 없음]·[평가 불가]는 뺌)
실험 평균	faithfulness_mean 등 RAGAS 4개	RAGAS 평균
실험 평균	tokens_in_mean · tokens_out_mean · latency_mean · latency_median	문항당 토큰, 응답 시간(초)
Observation Scores	Faithfulness · Context_precision · Context_recall · Answer_relevancy	문항별 RAGAS (목록에서는 평균으로 보임)
Observation Scores	Hit_at_k · Recall_at_k · Mrr_at_k	문항별 검색 점수. k마다 이름이 달라 k5는 _at_5, k10은 _at_10에만 값
Eval (서버 평가기)	Judge_facts 등	문항별 Judge 점수

_state가 붙은 열(©)은 켜지 않아도 됩니다. 점수 대신 채점됨 · 해당 없음 · 평가 불가 상태만 들어 있습니다.

2. 실험끼리 문항별로 비교하기

1. 실험 목록에서 비교할 실험(예: B k5 · k8 · k10)을 체크하고 **Compare**.
2. 첫 실험이 **Baseline**, 나머지가 **Comp**. 위쪽 SUMMARY에 지표별 평균과 차이, ↗ 오른 문항 수 · ↘ 내린 문항 수가 나옵니다.
3. SUMMARY는 '이 페이지(20문항)' 기준입니다. 30문항 전체 평균은 실험 목록의 *_mean 열을 보세요.
4. 아래로 내리면 문항마다 두 실험의 답변과 점수를 나란히 볼 수 있습니다.

3. 문항 하나의 토큰 · 시간 · 검색 결과 보기

1. Compare 화면이나 실험 문항 목록에서 문항을 누르면 trace 화면이 열립니다.
2. 맨 위에 전체 응답 시간과 토큰 합계, 왼쪽 단계 목록에 검색 → 답변 생성 순서가 나옵니다.
3. LLM 호출(**ChatOpenAI**, generation)을 누르면 입력 · 출력 · 캐시 토큰이 나뉘어 보입니다.
4. 오른쪽 Scores 탭에 그 문항의 검색 점수 · RAGAS · Judge(설명 포함)가 모두 붙어 있습니다.

4. 여러 trace의 토큰을 한꺼번에 보기

1. 왼쪽 메뉴 **Tracing** → **Observations**.

2. 필터: Type = GENERATION, 필요하면 시간 범위나 이름 조건을 더합니다.
3. Usage 열에 호출마다 입력 → 출력 토큰이 나옵니다.

> **비용** Langfuse 서버에 gpt-6-luna 가격이 등록되어 있지 않아 비용(Total Cost)은 0으로 나옵니다. 토큰 수는 정상입니다. 비용을 보려면 서버 관리자가 Settings → Models에 가격을 등록해야 합니다.

실험 다시 돌리기 · 점수 올리기

kant-rag 폴더, PowerShell 기준. 실험 하나(30문항)에 약 10분 걸립니다.

실험 하나

```
$env:PYTHONIOENCODING = "utf-8"; $env:OPENAI_MODEL = "gpt-6-luna"
.venv/Scripts/python.exe -W ignore evaluation/langfuse_experiment.py `
 --impl retrieval_b --difficulty basic --split development `
 --modes hybrid_rerank --top-k 15 --concurrency 2 --tag basic-dev-k15-gpt6luna
```

옵션	뜻
--impl	retrieval_a 또는 retrieval_b (B면 이름 앞에 b_)
--difficulty	basic · intermediate · advanced
--top-k	답변에 넣는 최종 문맥 수 k. 검색 점수도 이 k로 계산
--concurrency	동시 처리 문항 수. GPU 8GB에서는 2 권장 (모델 3개가 약 7GB 사용)
--tag	실험 이름 끝. {난이도}-dev-k{k}-gpt6luna 형식으로 맞춤
--no-ragas	RAGAS 채점 없이 빠르게 (검색 점수 · 서버 Judge만)

세 난이도 한 번에

```
foreach ($d in "basic","intermediate","advanced") {
 .venv/Scripts/python.exe -W ignore evaluation/langfuse_experiment.py --impl retrieval_b `
 --difficulty $d --split development --modes hybrid_rerank --top-k 15 --concurrency 2 `
 --tag "$d-dev-k15-gpt6luna"
}
```

실험 평균 점수 올리기 (정답률 · Judge 평균 · 토큰 · 응답 시간)

검색 · RAGAS 평균은 실험이 끝날 때 자동으로 올라갑니다. 서버 Judge는 문항마다 따로 붙어 실험 평균이 없으므로 아래로 모아 올립니다. 다시 실행해도 덮어씁니다.

```
.venv/Scripts/python.exe -W ignore artifacts/retrieval_a/cmp_eval/collect_k.py
.venv/Scripts/python.exe -W ignore artifacts/retrieval_a/cmp_eval/upload_run_summary.py collected_k.json --judge
```

1단계 검색 실험

```
.venv/Scripts/python.exe -W ignore artifacts/retrieval_a/cmp_eval/k_sweep.py retrieval_a advanced
# 검색기 · 난이도마다 따로 실행 권장 (GPU 메모리). 결과는 k_sweep.json에 이어서 기록
```